

CSC 480: Artificial Intelligence

3- Tree Search

Rodrigo Canaan
Assistant Professor
Computer Science Department
Cal Poly, San Luis Obispo
rcanaan@calpoly.edu

Aug 5, 2024

Monte-Carlo Tree Search (MCTS)

The problem with tree search

- Most tree search algorithms: $O(b^d)$
- Intractable for many problems
- Two approaches:
 - Reduce effective *breadth*
 - Reduce effective *depth*

b = breadth
d = depth

Node selection and pruning
Avoid expanding all nodes

Node evaluation
Cut search at a certain depth
with approximate value

How to improve search?

- Key ideas:
 - Replace heuristic evaluation function with random **rollouts** - reduces depth
 - Use a heuristic (**UCT value**) to choose which nodes to expand next based on an exploration/exploitation tradeoff - reduces breadth

Example: DeepBlue

- Reduces *breadth* with several heuristics for node selection and pruning
- Reduces *depth* with heuristics for node evaluation



Limitations

- Designing good expansion and evaluation heuristics can be hard!
- Many chess experts contributed to the heuristics in DeepBlue
- In certain problems (like the game of Go), the best human-crafted heuristics still didn't perform well

Key ideas in MCTS

- Evaluation heuristic can be replaced by a random *rollout*
 - Reduces effective *depth*
- Expand nodes balancing *exploration* of new nodes with *exploitation* of promising solutions
 - Asymmetrically reduces effective *breadth*
- No domain-specific heuristics needed!



Related problem: Multi-Armed Bandits

- ❖ Agent can pull one of n levers, each giving a reward from some random distribution
- ❖ How to maximize expected return over time?
- ❖ **Some ideas**
 - ❖ Pull at random (pure exploration)
 - ❖ Pull each lever k times, then only pull the best (exploitation)
 - ❖ Pull random lever with small probability, otherwise pull best lever
 - ❖ Is there a more principled approach?



Source: [Wikipedia](#) (CC BY-SA 3.0)

Which arm to pull?

Flat Monte Carlo

Share trials uniformly
between arms

ϵ -Greedy

$P(1 - \epsilon)$ – Best arm so far

$P(\epsilon)$ – Random arm

UCB1 (Auer et al (2002)).

Choose arm j so as to
maximise:

$$\bar{X}_j + \sqrt{\frac{2 \log n}{T_j(n)}}$$

Mean
so far

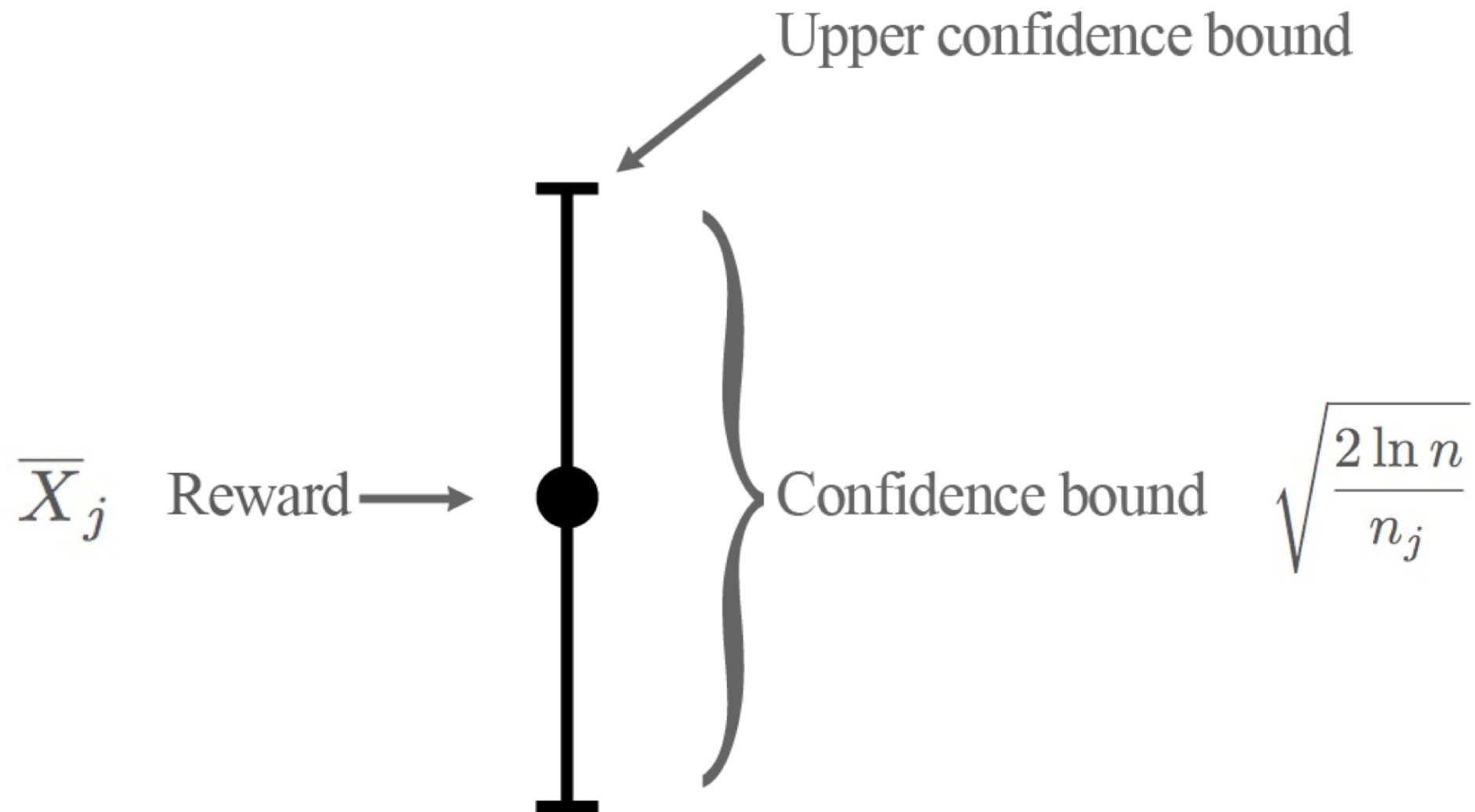
Upper bound
on variance

n = number of plays so far

$T_j(n)$ = number of times arm j was pulled

Confidence Bounds

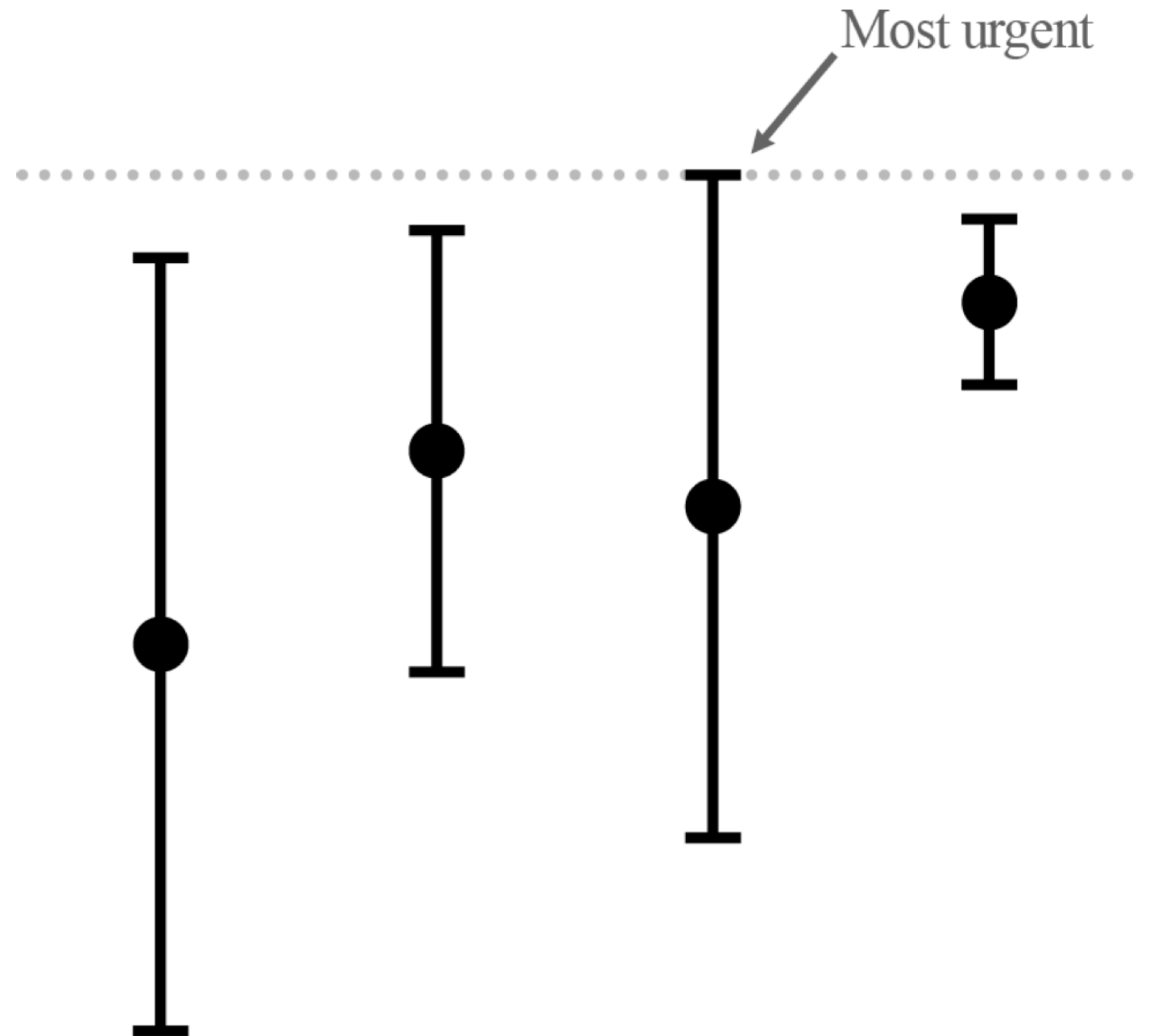
- Confidence in the reward's accuracy



- More visits = tighter bound

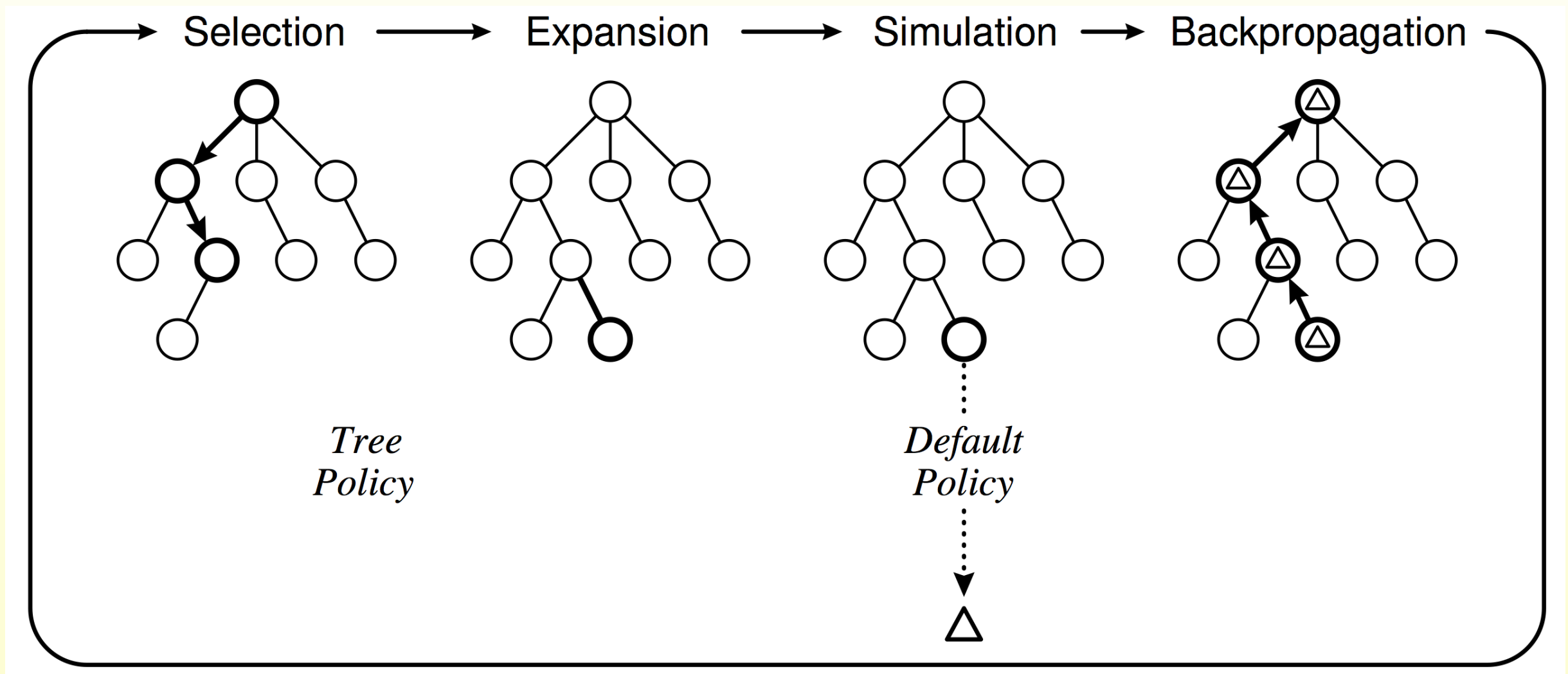
Most Urgent

- Most urgent node has the highest UCB
- **Not** highest reward
- **Not** widest spread



Browne, C. (n.d.). Monte Carlo Tree Search

MCTS Process

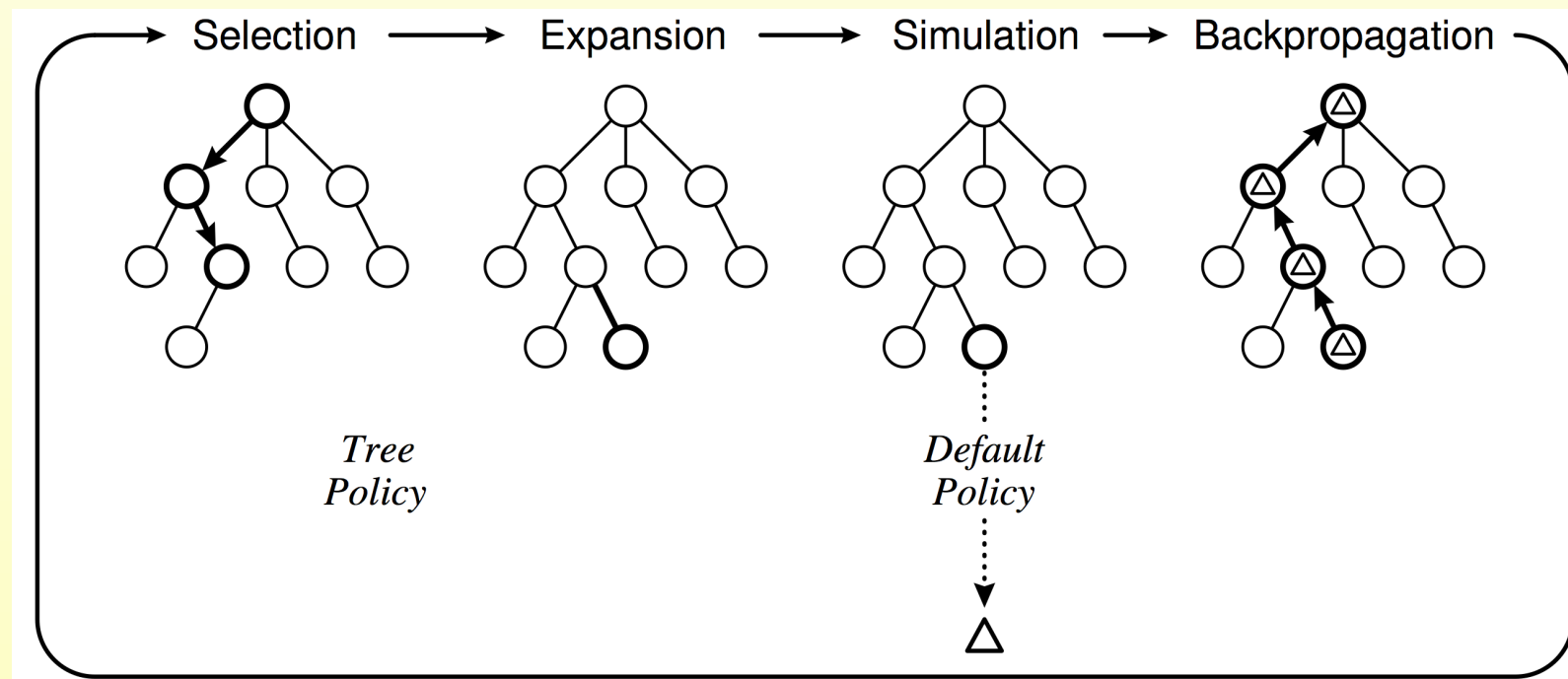


MCTS outline

- ❖ Iteratively build a tree from top to bottom
- ❖ Start each iteration by visiting the root
- ❖ If all children of visited node have been added to the tree:
 - ❖ Use UCB1 formula to choose which node recursively visit (Selection)
- ❖ If a node has some children not yet on the tree:
 - ❖ Add a random unexplored child node to the tree (Expansion)
 - ❖ Perform rollouts from that node to a terminal state (Default Policy)
 - ❖ Back propagate the final score of the rollout up the tree (Backpropagation)

MCTS outline

- ❖ Iteratively build a tree from top to bottom
- ❖ Start each iteration by visiting the root
- ❖ If all children of visited node have been added to the tree:
 - ❖ Use UCB1 formula to choose which node recursively visit (Selection)
- ❖ If a node has some children not yet on the tree:
 - ❖ Add a random unexplored child node to the tree (Expansion)
 - ❖ Perform rollouts from that node to a terminal state (Default Policy)
 - ❖ Back propagate the final score of the rollout up the tree (Backpropagation)



MCTS outline

- ❖ Iteratively build a tree from top to bottom
- ❖ Start each iteration by visiting the root
- ❖ If all children of visited node have been added to the tree:
 - ❖ Use UCB1 formula to choose which node recursively visit (Selection)
- ❖ If a node has some children not yet on the tree:
 - ❖ Add a random unexplored child node to the tree (Expansion)
 - ❖ Perform rollouts from that node to a terminal state (Default Policy)
 - ❖ Back propagate the final score of the rollout up the tree (Backpropagation)
- ❖ **Note: Tree Policy = Selection + Expansion**
 - ❖ Returns the next node to add to the tree and perform rollout from

MCTS - Pseudocode

Algorithm 1 General MCTS approach.

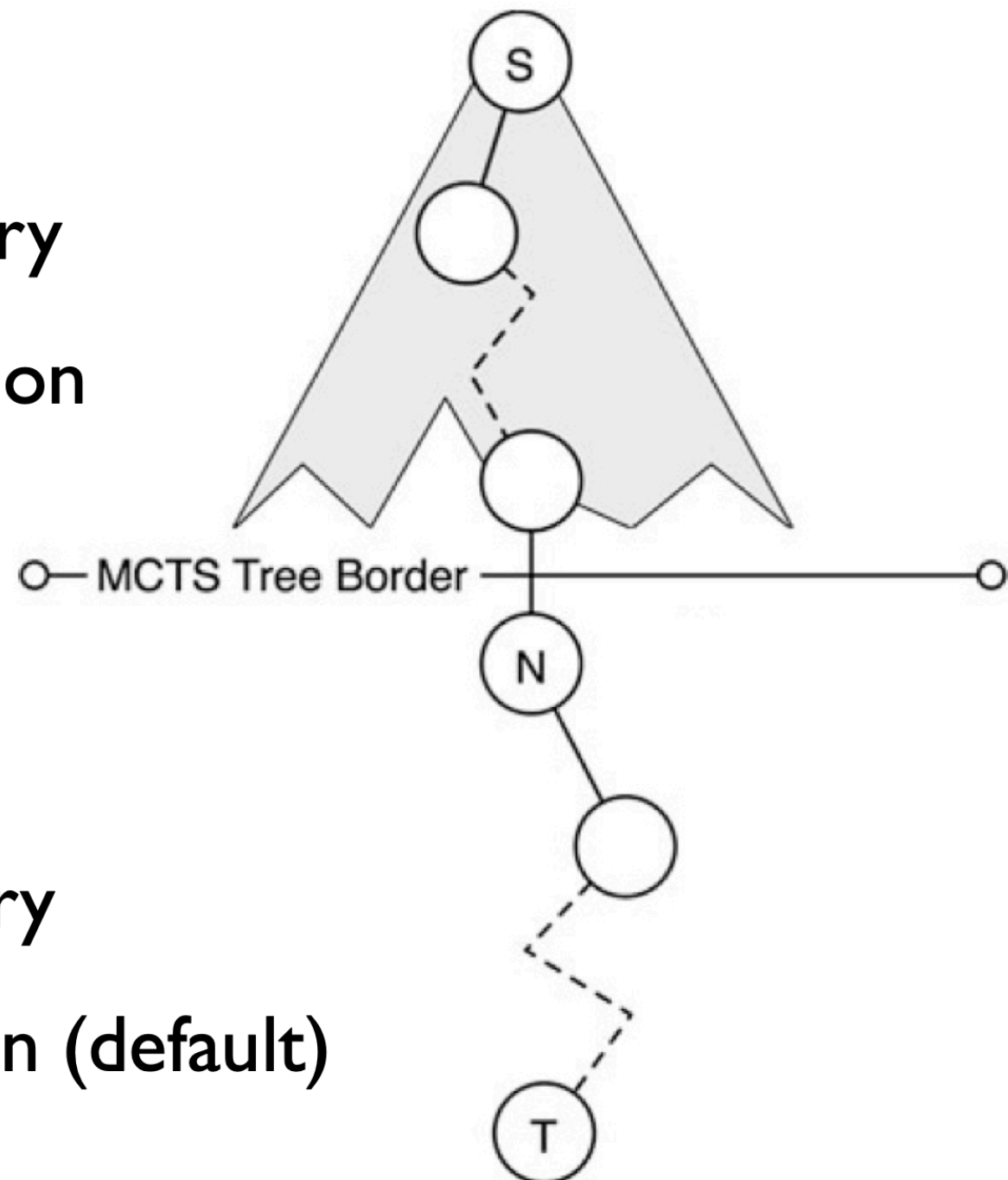
```
function MCTSSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
    BACKUP( $v_l, \Delta$ )  
  return  $a(\text{BESTCHILD}(v_0))$ 
```

By prof. Julian Togelius at NYU. Used with permission.
Original pseudocode on all slides by Browne, C. (n.d.). [Monte Carlo Tree Search](#)

Policies

Tree Policy

- Above the tree boundary
- Intelligent action selection



Default policy

- Below the tree boundary
- Random action selection (default)

Finnsen & Bjornsson (2008)

Tree policy

```
function TREEPOLICY( $v$ )  
  while  $v$  is nonterminal do  
    if  $v$  not fully expanded then  
      return EXPAND( $v$ )  
    else  
       $v \leftarrow$  BESTCHILD( $v, Cp$ )  
  return  $v$ 
```

Note that node selected for expansion does not need to be a leaf of the tree (the nonterminal test refers to the game state)

Tree expansion

```
function EXPAND( $v$ )  
  choose  $a \in$  untried actions from  $A(s(v))$   
  add a new child  $v'$  to  $v$   
    with  $s(v') = f(s(v), a)$   
    and  $a(v') = a$   
return  $v'$ 
```

Best child (UCT)

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD

$Q(v')$ = cumulative value of
child v'

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

$N(v')$ = number of visits of child v'

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

$N(v)$ = number of visits of current node v , = sum ($N(v')$) for all child v'

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD(v, c)

Q = cumulative value of child v'

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

$N(v)$ = number of visits of current node v , = sum ($N(v')$) for all child v'

$N(v')$ = number of visits of child v'

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

Exploitation factor
(average score of child)

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Best child (UCT)

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}}$

Exploration factor
(relative visits of child to its
siblings)

- Standard UCT equation (compare UCB)
- Higher values of c lead to more exploration

Default policy (rollout)

```
function DEFAULTPOLICY( $s$ )  
  while  $s$  is non-terminal do  
    choose  $a \in A(s)$  uniformly at random  
     $s \leftarrow f(s, a)$   
  return reward for state  $s$ 
```

- Each time a node is added to the tree, the default policy plays out until the terminal state of the game
- The standard is to do this uniformly randomly

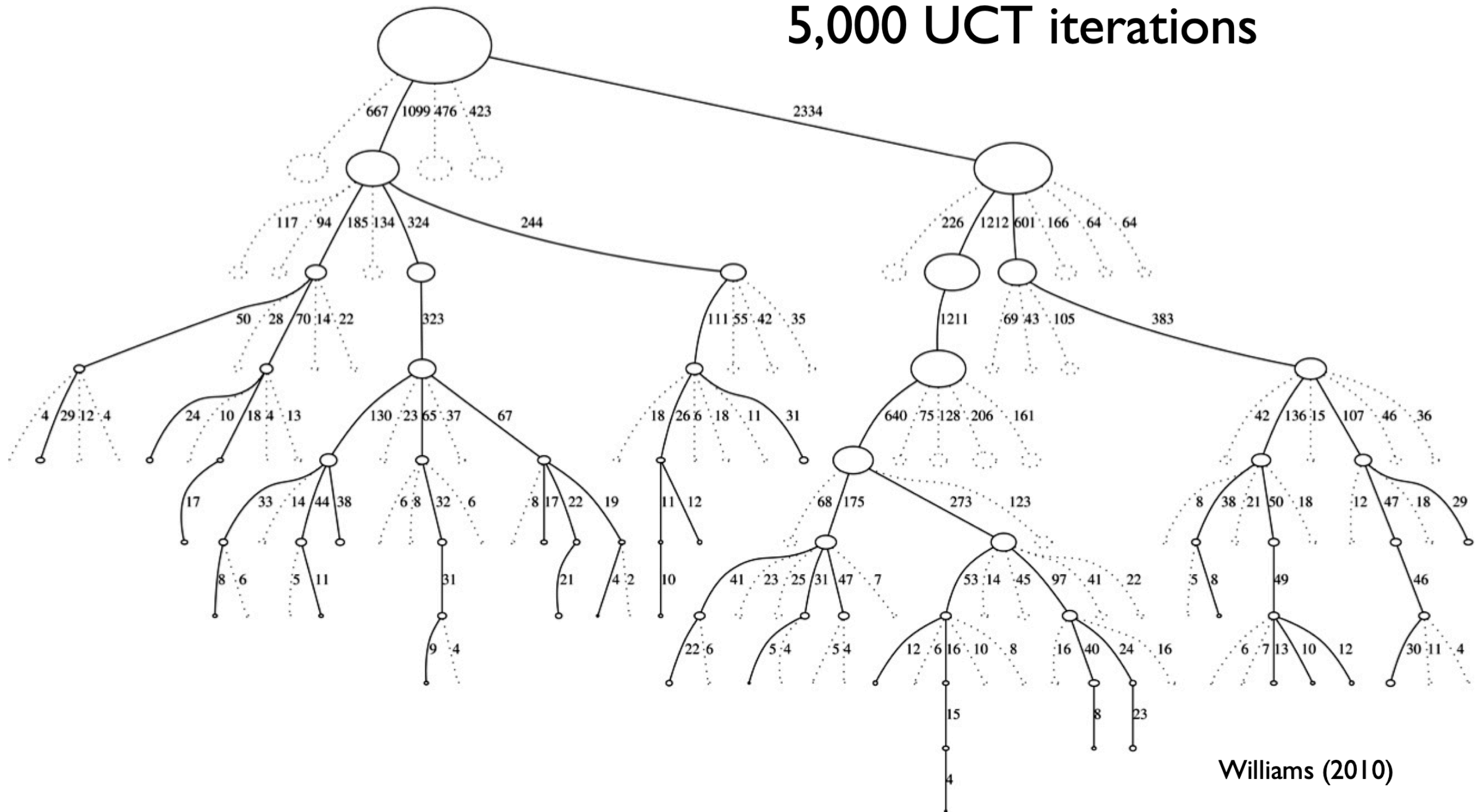
Backup

```
function BACKUP( $v, \Delta$ )  
  while  $v$  is not null do  
     $N(v) \leftarrow N(v) + 1$   
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$   
     $v \leftarrow$  parent of  $v$ 
```

- v is the new node added to the tree by the tree policy
- Back up the values from the added node up the tree to the root

MCTS Tree Growth

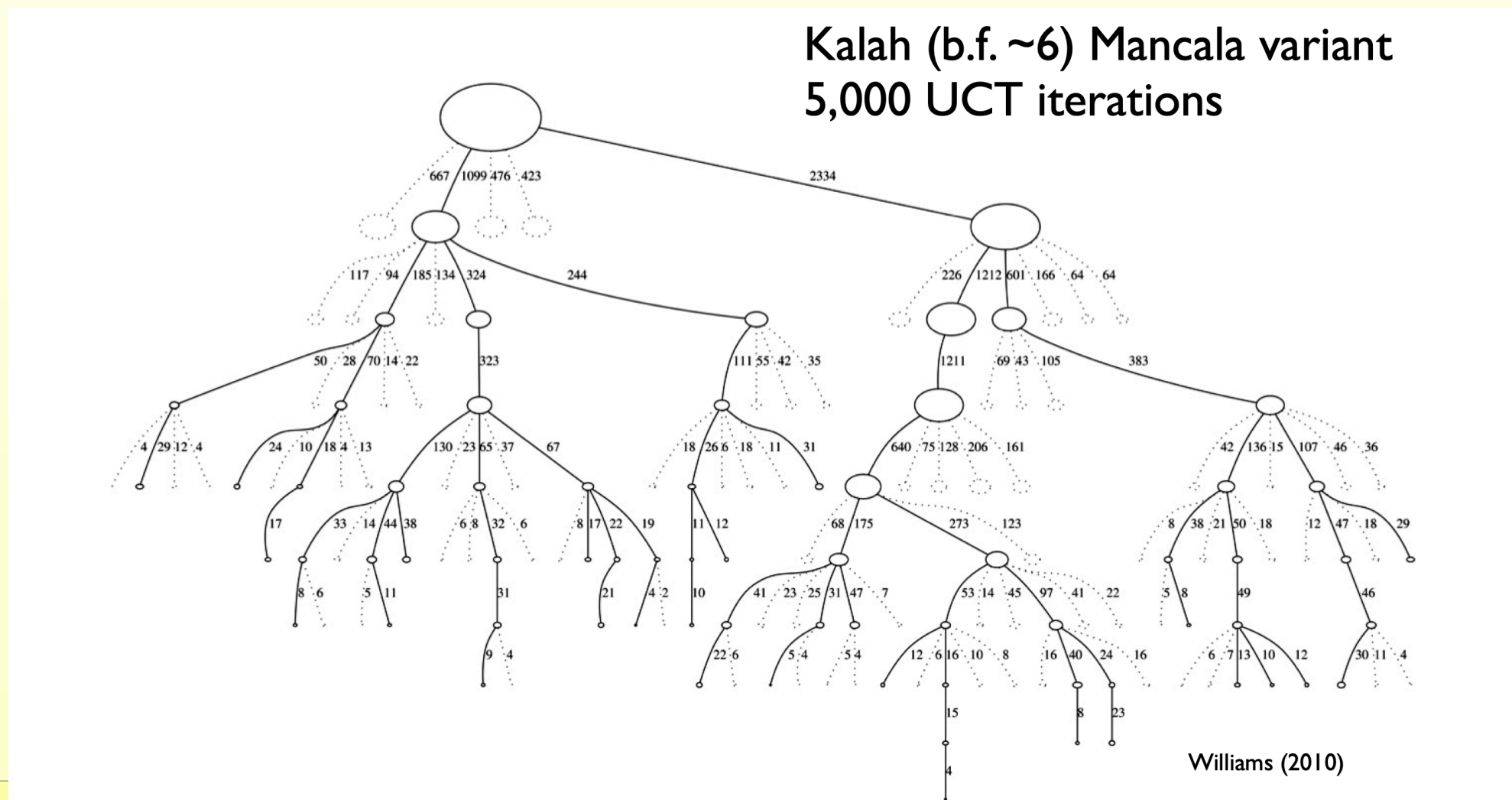
Kalah (b.f. ~6) Mancala variant 5,000 UCT iterations



Williams (2010)

MCTS Tree Growth

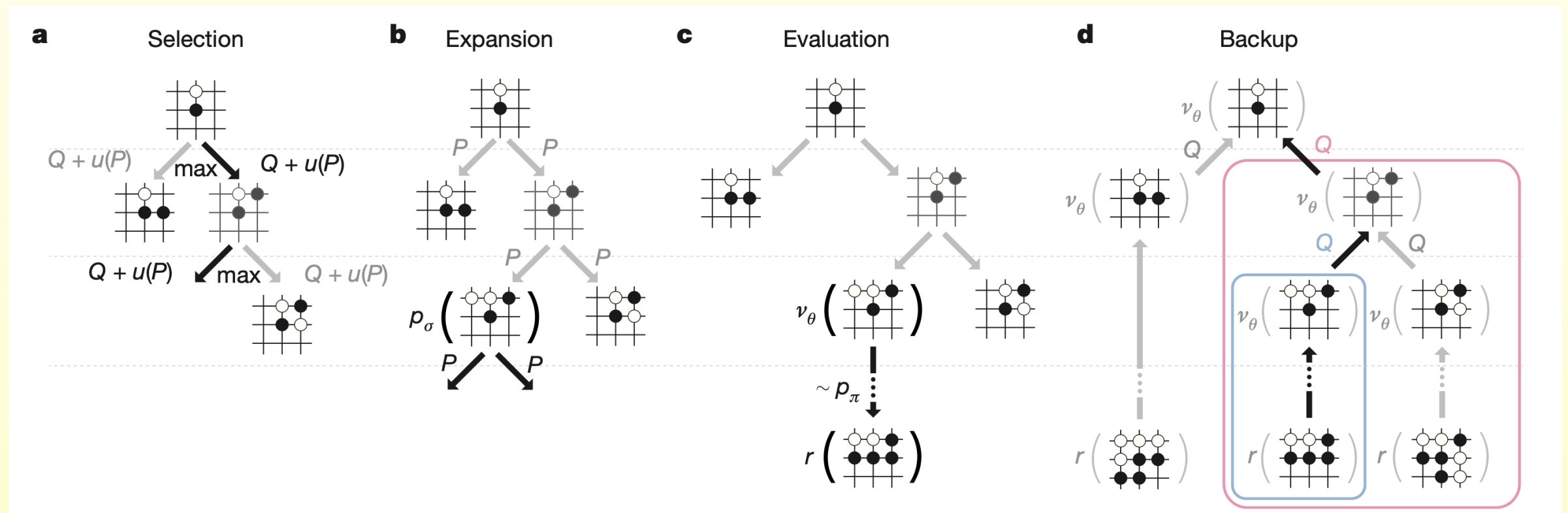
- ❖ search focusses on more interesting parts of the tree
 - ❖ Search is Asymmetric
- ❖ “anytime” graceful exit
 - ❖ search can be stopped anytime and delivers the best outcome so far



Browne, C. (n.d.). Monte Carlo Tree Search Retrieved October 20, 2021

Alpha Go

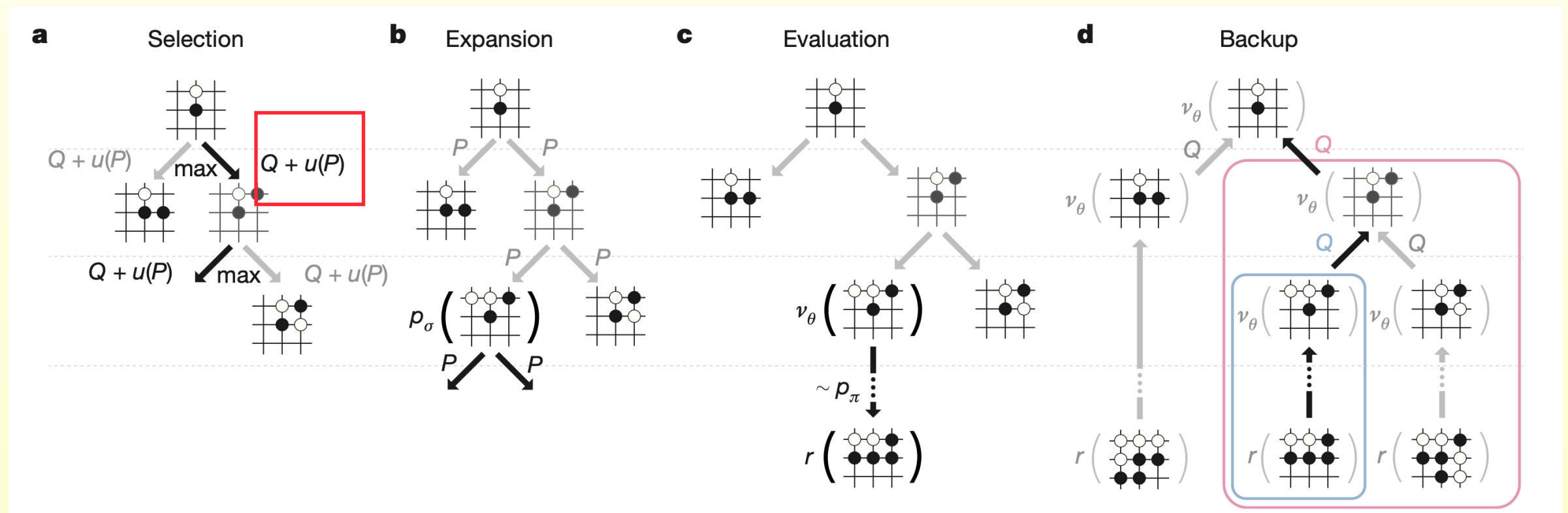
- ❖ Combines MCTS with trained neural networks
- ❖ During selection, a modified UCB formula has the “exploration factor” combined with the output of a “policy” neural network.



- ❖ During rollouts, the value of the rollout is averaged with the output of a “value” neural network

Alpha Go

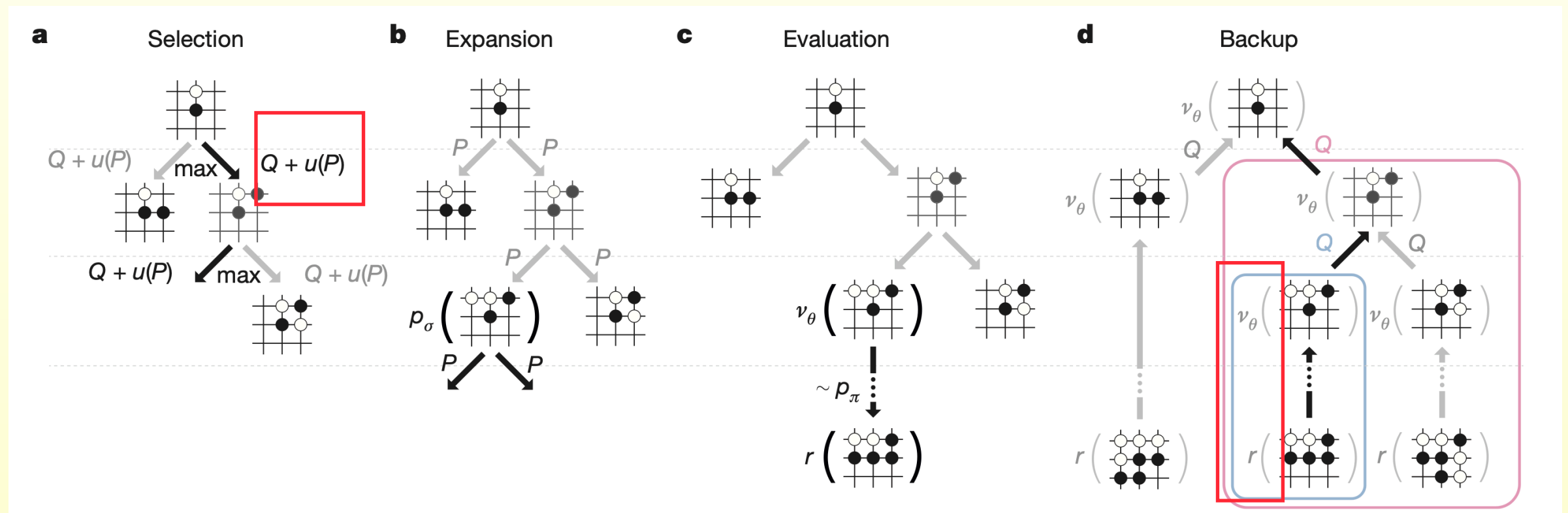
- ❖ Combines MCTS with trained neural networks
- ❖ During selection, a modified UCB formula has the “exploration factor” combined with the output of a “policy” neural network.



- ❖ During rollouts, the value of the rollout is averaged with the output of a “value” neural network

Alpha Go

- ❖ Combines MCTS with trained neural networks
- ❖ During selection, a modified UCB formula has the “exploration factor” combined with the output of a “policy” neural network.



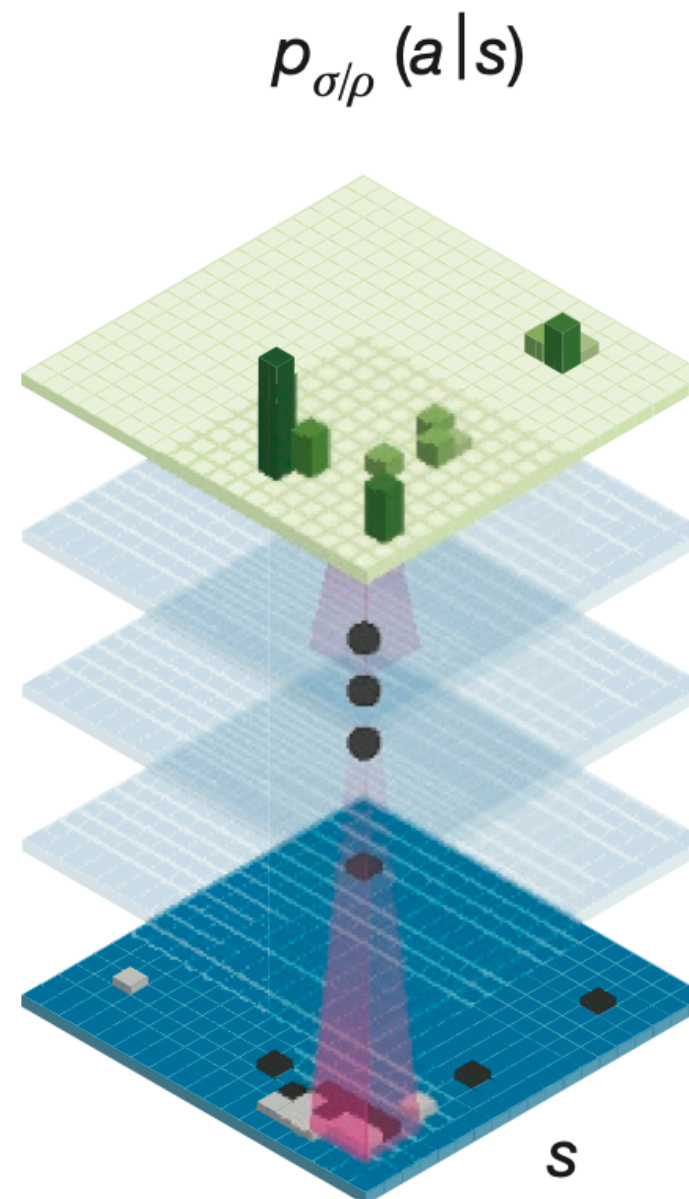
- ❖ During rollouts, the value of the rollout is averaged with the output of a “value” neural network

Alpha Go

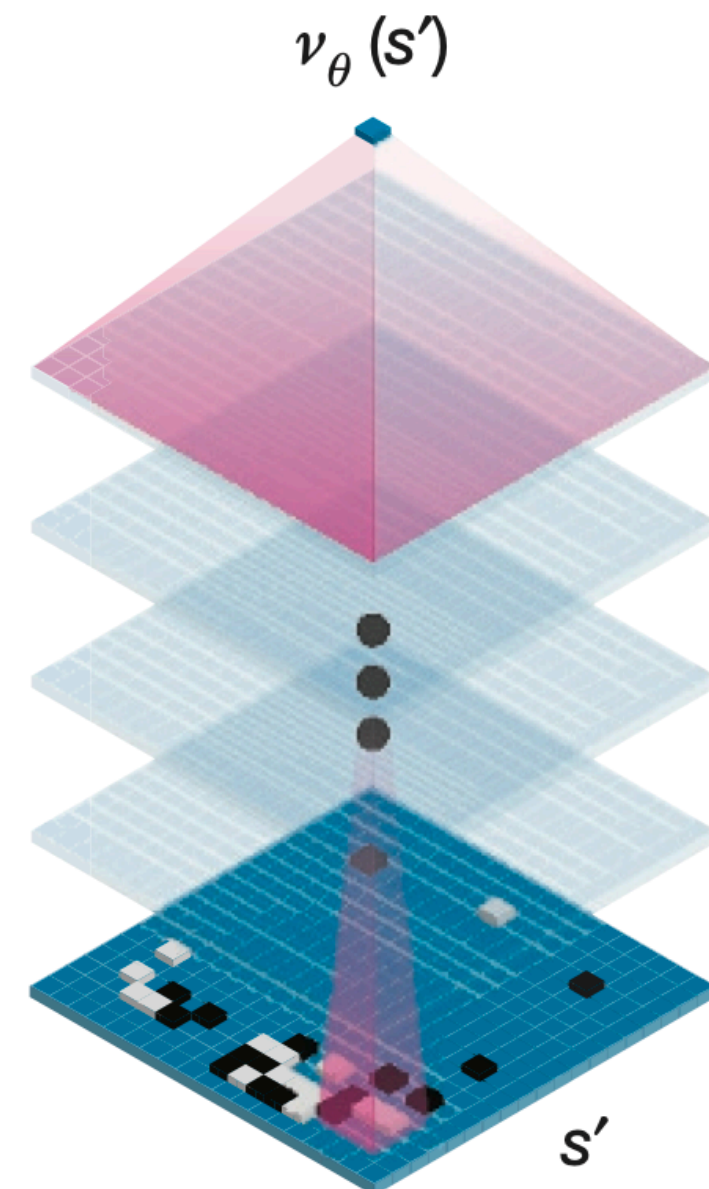
- ❖ Policy and Value network are kickstarted by **supervised learning** from human demonstrations
- ❖ They are then further trained by **reinforcement learning** via self-play

b

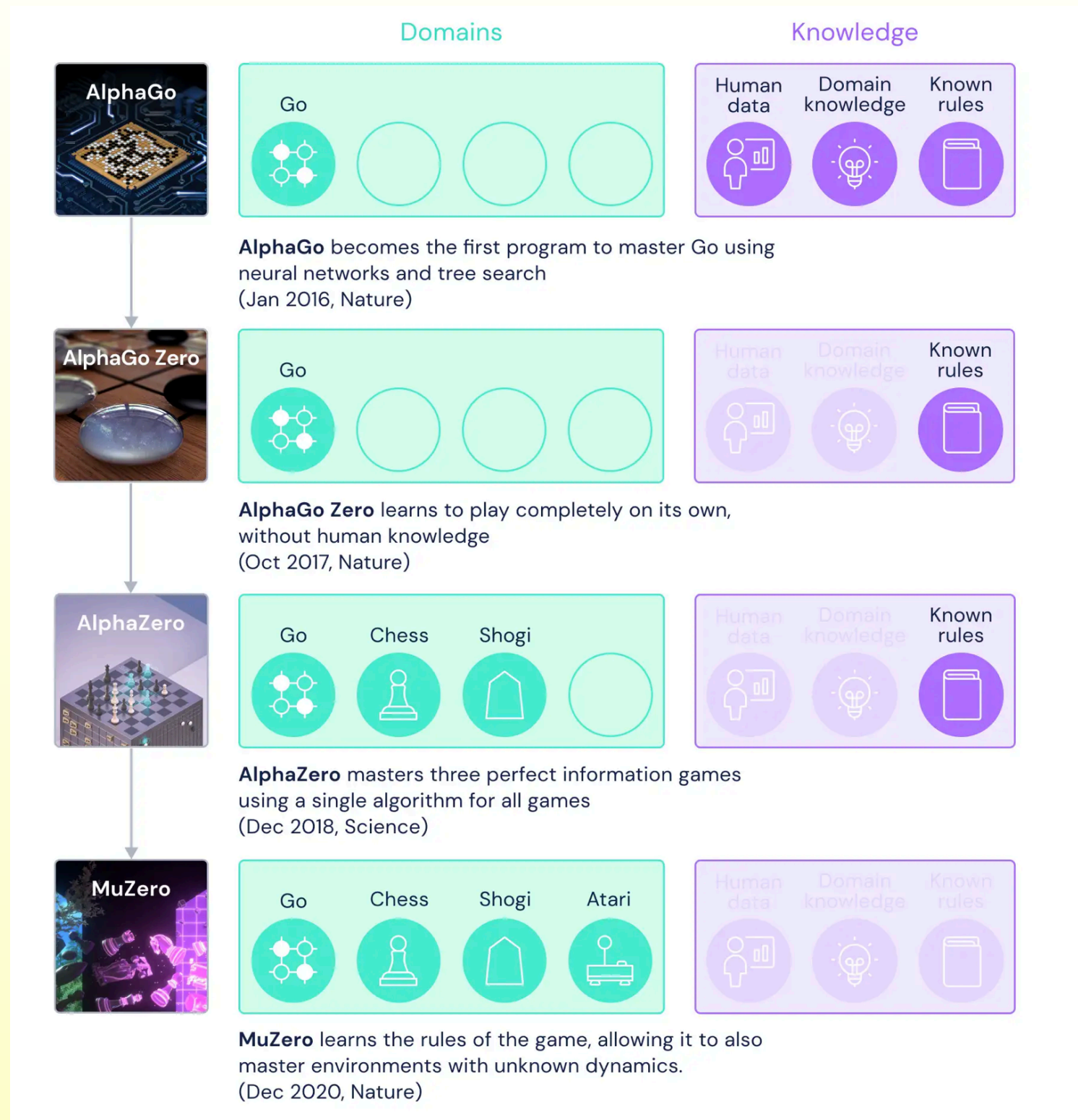
Policy network



Value network



Successors of Alpha Go



Source: MuZero: Mastering Go, chess, shogi and Atari without rules
(DeepMind Blog, access Oct 21, 2021)

Successors of Alpha Go

❖ Alpha Go Zero:

- ❖ Single neural network combines the role of policy and value networks
- ❖ Trained via self-play alone, no human demonstrations
- ❖ Does not use rollouts (relies purely on network for value estimation)

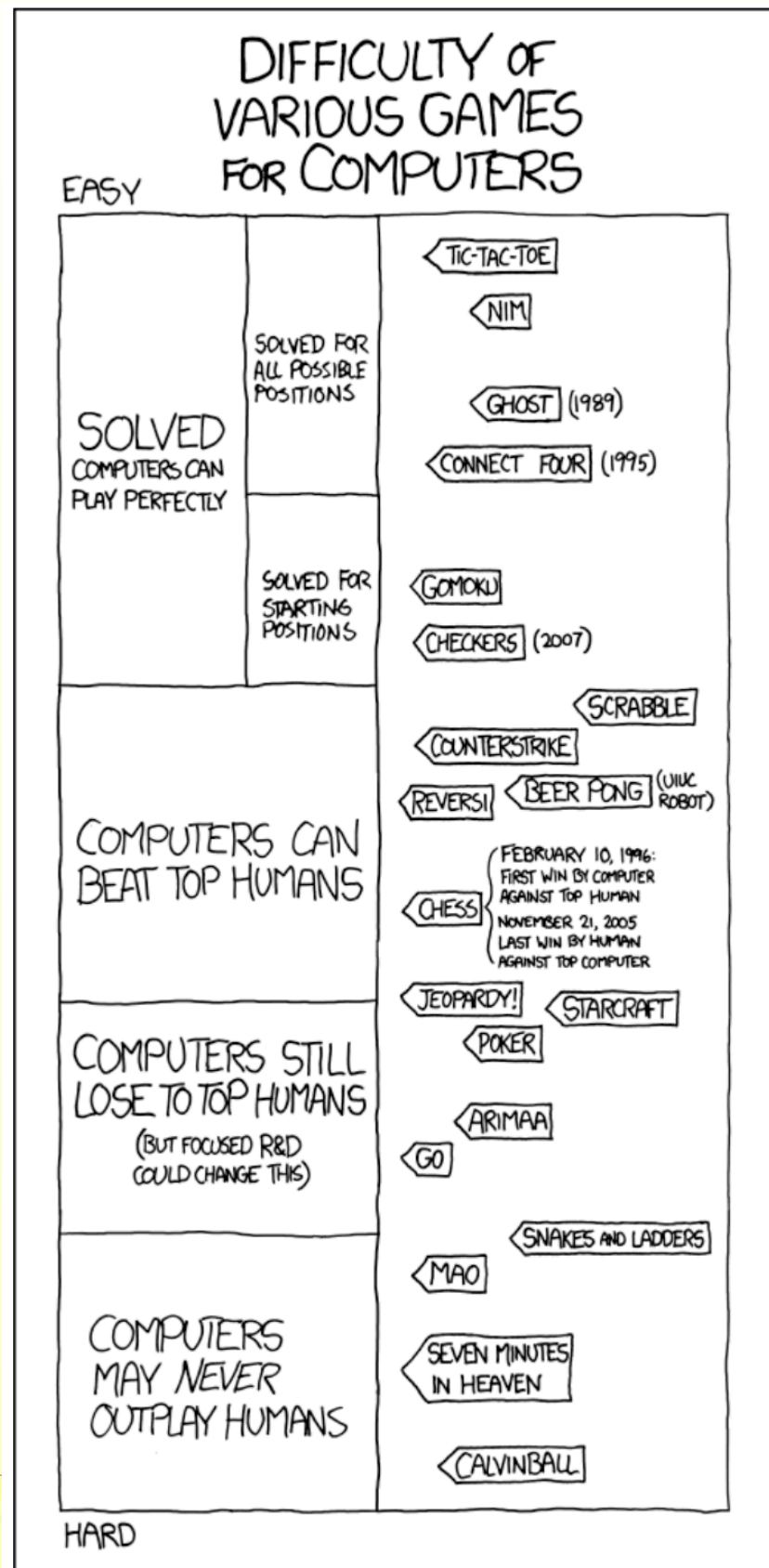
❖ Alpha Zero:

- ❖ Similar to Alpha Go Zero, but achieves superhuman performance on various games
 - ❖ Go
 - ❖ Chess
 - ❖ Shogi
- ❖ Trained individually for each of these game types

❖ Mu-Zero

- ❖ Does not know game rules in advance
- ❖ Does not use an explicit forward model
- ❖ Environment dynamics are also learned and modeled by a neural network
- ❖ Also plays Atari

XKCD 1002



❖ A bit outdated now...

- ❖ Go and Poker bots have beaten top humans
- ❖ Starcraft (and Dota2) bots have beaten professional humans
 - ❖ Not too consistently
 - ❖ With some limitations

MCTS Takeaways

- ❖ **Applicable to vastly larger state spaces than most other methods**
- ❖ **Does not require heuristics**
 - ❖ But heuristics can be incorporated
- ❖ **“Anytime” graceful exit**
 - ❖ search can be stopped anytime and delivers the best outcome so far
- ❖ **Transitions and probabilities do not need to be known in advance**
 - ❖ As long as a “forward model” (simulator) is available
- ❖ **Drawback: requires lots of memory**
 - ❖ Every node must be kept in memory
- ❖ **Drawback: no guarantee of completeness or optimality**